

A Digital Twin Modeling Code Generation Framework based on Large Language Model

1st Jiabao Dong

School of Automation Science and Electrical Engineering
Beihang University
Beijing, China
aarondong@buaa.edu.cn

2st Lei Ren

School of Automation Science and Electrical Engineering
Beihang University
Beijing, China
renlei@buaa.edu.cn

Abstract—Digital twin has made significant achievements in applications across various fields by bridging the gap between virtual and physical world. However, the modeling of digital twin faces challenges of labor-intensive and time-consuming manual modeling or coding. With the emergence of artificial intelligence foundation models, the integration of large language models (LLMs) and digital twin may contribute to enhancing modeling efficiency significantly. Therefore, in this paper, a framework of digital twin modeling code generation is proposed based on LLMs. Given instructions in natural language prompts, modeling code in digital twin software will be generated from LLMs. Firstly, a tree diagram-based code generation method is proposed to generate main section of modeling code. Additionally, a randomized code completion method is proposed to complete missing parts which are unspecified or unclear in prompts. Finally, a case study is given in NVIDIA Omniverse for a digital twin construction based on framework proposed in this paper.

Index Terms—digital twin, large language model, modeling and simulation, code generation, AI foundation model

I. INTRODUCTION

With the development of information and communication technologies, one of the most promising technologies, digital twin, establishes a virtual representation of real world to bridge the gap between digital and real world. Since the definition was proposed, digital twin has made significant contributions in applications across various fields such as smart cities, automotive, healthcare, manufacturing, etc. In these domains, digital twin technology enables more informed decision-making processes with enhanced efficiency and innovation [1].

Typically, the implementation steps of digital twin involves modeling, data integration, simulation, analysis and real-time feedback [2]. Among them, one of the most crucial steps in digital twin construction is the modeling of the target objects. The ability to create precise and realistic virtual models mirroring physical entities is fundamental to the following

implementation steps of digital twin technology [3]. However, the process of digital twin modeling faces some challenges summarized as follows.

- (1) **Manual labor-intensive modeling**: Current modeling software offers a method of manual manipulation, allowing users to create detailed models by drag-and-drop and interactive modeling. However, this process is often labor-intensive and time-consuming.
- (2) **High level of expertise for coding**: Alternatively, some software enables users to generate modeling scenes directly by coding and debugging. While this approach can be more efficient, it requires a high level of expertise, making it difficult for newcomers to adopt.
- (3) **Hallucinations caused by AIGC models**: Emerging technologies empowered by artificial intelligence generated content (AIGC) technologies, like text-to-3D, allow for the rapid generation of 3D models through generative models [4]. However, these models lack the necessary constraints of real-world physical principles. Hallucinations will be caused by unsuitable generation in practical applications such as industrial simulation [5], [6].

By addressing the limitations of current modeling methods, the valuable potential of digital twin will be realized with accuracy and efficiency of model generation. With the emergence of artificial intelligence (AI) foundation models [7], compared with previous AI models [8], the large language models (LLMs) [9], [10] are capable of producing human-like text, such as text, translation, question answering and code generation, based on given prompts [11], [12]. Therefore, this paper proposes a solution to overcome existing challenges with integration of LLMs and digital twin modeling. The LLMs will generate modeling code required by users with natural language prompts. The generated code will be utilized in digital twin software for digital scene models construction. The main contributions are listed as follows.

- (1) In this paper, a framework of digital twin modeling code generation is proposed based on LLMs. This framework requires users to simply describe their modeling require-

The research is supported by the National Key Research and Development Program of China No. 2023YFB3308000, the NSFC (National Science Foundation of China) project No. 62225302, 92167108, 62173023, and the Academic Excellence Foundation of BUAA for PhD Students.

Corresponding Author: Lei Ren.

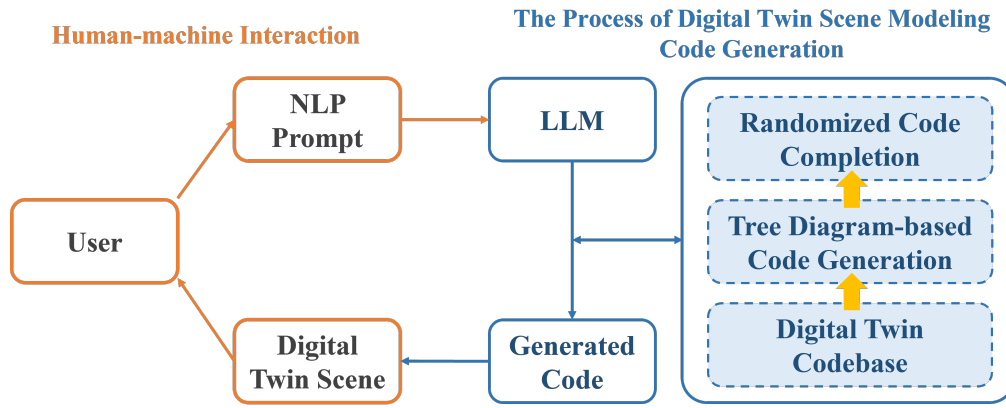


Fig. 1. Framework of digital twin modeling code generation based on LLMs.

ments using natural language prompts. The LLMs intelligently understands then allows for the smart, accurate, and rapid generation of modeling code that can be directly compiled and executed in digital twin 3D modeling and simulation software.

- (2) Additionally, a tree diagram-based code generation method is proposed to generate main section of modeling code. Leveraging the natural language understanding capabilities of LLMs, this method effectively extracts user modeling requirements from a hierarchical code generation mechanism to accurately translate user modeling needs into digital twin modeling code.
- (3) Finally, in consideration of missing parts which are unclear or not specified in prompts from users, a randomized code completion method is proposed to complete missing parts in generated code. In this completion method, parameters are classified into continuous and discrete for corresponding completion strategy.

II. FRAMEWORK OF DIGITAL TWIN MODELING CODE GENERATION BASED ON LLMs

This paper proposes a framework for generating digital twin modeling code based on LLMs. This framework requires users to simply describe their modeling requirements using natural language prompts. The LLMs will intelligently understand these requirements, allowing for the smart, accurate, and rapid generation of modeling code that can be directly compiled and executed in 3D modeling and simulation software.

Additionally, users can further refine the generated modeling scenes by issuing new adjustment requirements through natural language commands. The LLMs model adjusts the generated modeling code accordingly, aiming to achieve modeling results that satisfy the users. As illustrated in Fig. 1, this method based on LLMs for digital twin modeling code generation can be mainly sequentially outlined as tree diagram-based code generation and randomized code completion. The framework of digital twin modeling code generation based on LLMs facilitates automatic digital twin modeling within modeling software with enhanced efficiency.

III. TREE DIAGRAM-BASED CODE GENERATION METHOD

To facilitate the logical and structured establishment of a modeling code generation process, this paper defines a tree-like diagram structure for describing the relationships among classes, functions, and parameters in generated modeling code.

In the tree-like diagram structure illustrated by Fig. 2, classes are the top-level elements representing abstract templates of objects. Each class may contain multiple functions, which are methods of the class used to perform specific operations. Each function can accept multiple parameters responsible for passing data or information to the function. Different classes are assigned to different modeling objects, each with its respective functions and parameters.

Different class structures often contain different functions, and each function may have distinct parameters. For instance, when importing an object during the digital twin modeling process, a tree-like diagram structure is utilized like Fig. 3. Following the defined tree structure, when importing an object, the class of the object will be first declared. Then, within this class, several functions to describe the object's geometric properties, appearance, and category will be defined. Subsequently, different parameters are assigned to different functions. For example, functions describing geometric properties may allocate parameters such as the Cartesian coordinates of the 3D object's position, Euler angles for rotation, and scaling factors indicating the object's size. Similarly, appearance-related functions may assign parameters such as material, texture and map. Category functions allocate different usage categories, material categories and shape categories to the object.

Based on this defined tree-like diagram structure, each potential object can be assigned corresponding classes, functions, and associated parameters. This structure also facilitates mapping between user input in natural language and the different classes, functions, and parameters in the modeling code. With this mapping manner, it is crucial for training LLMs to enhance their understanding and extraction capabilities of user modeling requirements from natural language prompt inputs, thereby generating more intelligent modeling code. This

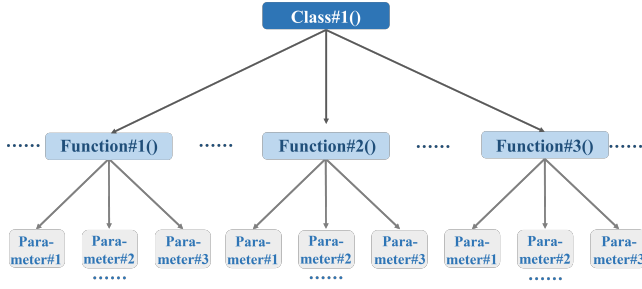


Fig. 2. A tree-like diagram structure for digital twin modeling code generation.

method not only streamlines the modeling process but also enhances user interaction by allowing iterative adjustments to modeling scenarios through natural language instructions, further refining the generated modeling outcomes.

To economize on training time and costs, the study envisions a pre-trained LLM as fundamental model for modeling code generation. During pre-training, the LLM learns from extensive internet text data to learn language statistical properties, grammar structures, and semantic meanings. This enables the LLM to effectively and accurately extract modeling requirements from user-input natural language, comprehending relationships between words, phrases, and sentences. Upon encountering input text data, initial processing involves encoding the input text into vector representations while retaining contextual information to enhance understanding of command meanings. Addressing contextual dependencies within natural language, the LLM will capture long-range dependencies and context, especially focusing on user-inputted modeling demands. Incorporating a hierarchical generation structure tailored to the aforementioned tree-like diagram structure, the code generation approach will first generate appropriate classes at the primary level. Next, suitable functions at the secondary level, and ultimately appropriate parameters at the tertiary level will be generated.

IV. RANDOMIZED CODE COMPLETION METHOD

In consideration of missing parts which are unclear or not specified in prompts from users, a randomized code completion method is needed.

Considering expectations and usage habits with LLMs for generating digital twin modeling code, users typically specify only the parts of the scene of interest, such as critical components like robotic arms, grippers, component appearances, and workstations in manufacturing tasks. Consequently, relying only on specified scene requirements often results in incomplete industrial scene generation. Apart from user-specified parts, numerous other aspects like backgrounds and lighting in a comprehensive digital twin scene also require modeling code. Furthermore, user-provided modeling requirements often lack precise values. For instance, when user input prompt 'put the object on the end of conveyor belt', the terms like 'end' in natural language are ambiguous in modeling code,

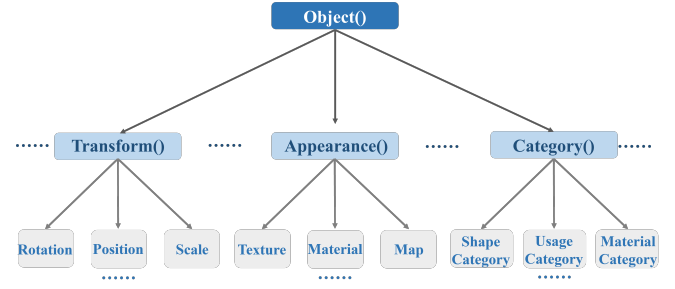


Fig. 3. An example of tree-like diagram structure for an object class in digital twin modeling code generation.

necessitating further specification for recognition in modeling code.

Therefore, this paper proposes a randomized modeling code completion method to generate unspecified or unclear parts in modeling scenes. Despite its randomized nature, outputs adhere strictly to real-world conditions. For instance, surface light sources or line light sources will be selected rather than point light sources in industrial assembly lines. Leather or metal materials will be selected over plastics or textiles for conveyor belt surfaces. Addressing situations where modeling objectives are unspecified or ambiguously described in user input, this method divides modeling parameters into discrete and continuous types, employing specific methods for each.

- (1) **Discrete Parameters:** For discrete parameters in modeling objectives, which are constrained to discrete values (e.g., types of light sources), this study employs sampling methods following a multinomial distribution. The probability density function is described as follows.

$$f(N_{1+k}) = \frac{1}{N}, 1 \leq k \leq N_2 - N_1 \quad (1)$$

- (2) **Continuous Parameters:** For continuous parameters in modeling objectives, which are distributed within a defined range (e.g., light color temperature), this study employs a sampling method based on a continuous uniform distribution. The probability density function for selecting values within the continuous range is defined as follows.

$$P(X_i = n_i) = \begin{cases} \frac{n!}{n_1! \dots n_k!} p_1^{n_1} \dots p_k^{n_k} & , \sum_{j=1}^k n_j = n \\ 0 & , \text{otherwise} \end{cases} \quad (2)$$

V. CASE STUDY

In this work, NVIDIA Omniverse is selected as the 3D modeling and simulation platform to realize a case study for industrial digital twin scene modeling [13] for several key reasons.

- (1) **Python Environment:** NVIDIA Omniverse provides a Python scripting environment, allowing users to write code to implement modeling and simulation functionalities. This facilitates the integration of open-source and third-party Python libraries to assist in the code modeling process.

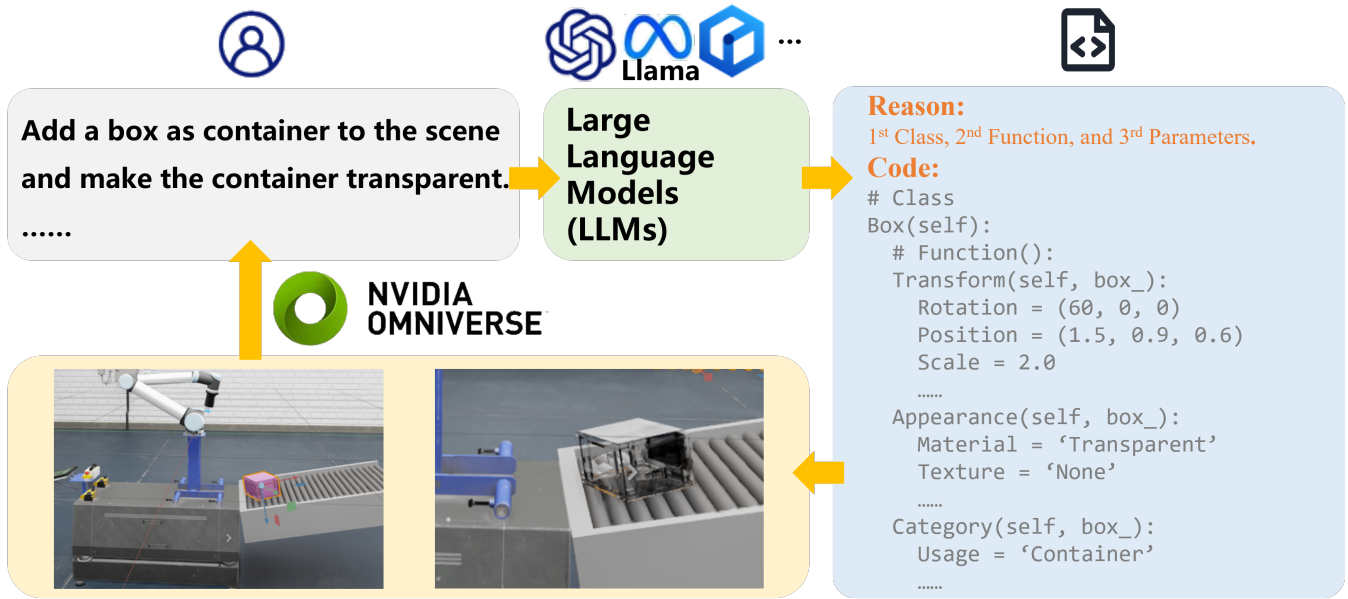


Fig. 4. Modeling code generation process for NVIDIA Omniverse.

- (2) **High-Fidelity Industrial Environments:** NVIDIA Omniverse offers multiple highly realistic industrial equipment and environments across various industrial applications to ensure trustworthiness of modeling and simulation.
- (3) **Visual Excellence:** Omniverse employs advanced techniques such as ray tracing, physical material and advanced lighting simulation, and high-performance GPU-accelerated rendering, providing users with better immersive visual effects.

Therefore, a case study is implemented in NVIDIA Omniverse. The interaction process is illustrated in Fig. 4.

VI. CONCLUSION

In this paper, a framework of digital twin modeling code generation is proposed based on LLMs, which requires users to simply describe their modeling requirements using natural language prompts. The LLMs will intelligently understand then allows for the smart, accurate, and rapid generation of modeling code that can be directly compiled and executed in digital twin 3D modeling and simulation software. Specifically, a tree diagram-based code generation method and a randomized code completion method is proposed to generate main section and complete missing parts in generated code. Future works may associate with enhancing generalization of more modeling objects and exploring in other modeling software.

REFERENCES

- [1] L. Ren, J. Dong, D. Huang, and J. Lü, "Digital twin robotic system with continuous learning for grasp detection in variable scenes," *IEEE Transactions on Industrial Electronics*, 2023.
- [2] D. Bishara, Y. Xie, W. K. Liu, and S. Li, "A state-of-the-art review on machine learning-based multiscale modeling, simulation, homogenization and design of materials," *Archives of computational methods in engineering*, vol. 30, no. 1, pp. 191–222, 2023.
- [3] X. Liu, D. Jiang, B. Tao, F. Xiang, G. Jiang, Y. Sun, J. Kong, and G. Li, "A systematic review of digital twin about physical entities, virtual models, twin data, and applications," *Advanced Engineering Informatics*, vol. 55, p. 101876, 2023.
- [4] C.-H. Lin, J. Gao, L. Tang, T. Takikawa, X. Zeng, X. Huang, K. Kreis, S. Fidler, M.-Y. Liu, and T.-Y. Lin, "Magic3d: High-resolution text-to-3d content creation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 300–309.
- [5] V. Rawte, A. Sheth, and A. Das, "A survey of hallucination in large foundation models," *arXiv preprint arXiv:2309.05922*, 2023.
- [6] Z. Wang, C. Lu, Y. Wang, F. Bao, C. Li, H. Su, and J. Zhu, "Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [7] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *arXiv preprint arXiv:2307.06435*, 2023.
- [8] L. Ren, J. Dong, X. Wang, Z. Meng, L. Zhao, and M. J. Deen, "A data-driven auto-cnn-lstm prediction model for lithium-ion battery remaining useful life," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 5, pp. 3478–3487, 2020.
- [9] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.
- [10] R. Dong, C. Han, Y. Peng, Z. Qi, Z. Ge, J. Yang, L. Zhao, J. Sun, H. Zhou, H. Wei *et al.*, "Dreamllm: Synergistic multimodal comprehension and creation," *arXiv preprint arXiv:2309.11499*, 2023.
- [11] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, and L. Zhang, "Exploring and evaluating hallucinations in llm-powered code generation," *arXiv preprint arXiv:2404.00971*, 2024.
- [12] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. Wang, and X. V. Lin, "Lever: Learning to verify language-to-code generation with execution," in *International Conference on Machine Learning*. PMLR, 2023, pp. 26 106–26 128.
- [13] L. Ren, J. Dong, L. Zhang, Y. Laili, X. Wang, Y. Qi, B. H. Li, L. Wang, L. T. Yang, and M. J. Deen, "Industrial metaverse for smart manufacturing: Model, architecture, and applications," *IEEE Transactions on Cybernetics*, 2024.